

Modélisation avancée des Systèmes d'Information

08 - Factory

Mohamed ZAYANI

2025/2026

Définition du « Factory »

- Le patron de conception Factory fait partie des **patrons de création**.
- Il est utilisé pour **déléguer** la création des objets à des sous-classes ou à des méthodes, ce qui permet de **découpler** le code client (**le code d'utilisation des objets**) du code de leur implémentation concrète.
- **Objectif principal:**
 - ❖ Centraliser et abstraire la création d'objets pour améliorer la flexibilité du code..
- **Problématique:**
 - Ne pas vouloir que le code client connaisse les classes concrètes à instancier.
 - Eviter le couplage fort entre le code qui utilise l'objet et le code qui le crée.
 - Anticiper des changements ou extensions (ajout de nouvelles classes) sans devoir modifier tout le code..
 - **Exemple sans Factory:**

```
// Couplage fort : le client doit connaître les classes concrètes
Forme forme = new Cercle();
forme.dessiner();
```

- Si demain on remplace Cercle par Triangle, il faut modifier le code partout où Cercle est instancié !!!!

Solution proposée par Factory

- **Déléguer** la création des objets à une classe spéciale (la Factory).
- Le client **ne connaît pas la classe concrète**, il ne dépend que de l'interface ou de l'abstraction.
- **Exemple avec Factory:**

```
// Couplage faible : le client ne connaît pas les classes concrètes  
Forme forme = FormeFactory.getForme("cercle");  
forme.dessiner();
```

• Maintenant, si on remplace l'implémentation de "cercle" dans la factory, le client ne change pas !

- **Principe:**

- Le client ne crée pas directement les objets via **new**.
- Il demande à une "factory" de lui fournir un objet qui respecte une interface ou une classe abstraite.
- La factory décide quelle classe concrète instancier.

- **Avantages :**

- **Faible couplage** : Le client dépend uniquement d'une interface ou classe abstraite, pas des classes concrètes.
- **Centralisation de l'instanciation**: Toute la logique de création est contenue dans une seule classe (la factory), facilitant la maintenance.
- **Respect des principes SOLID (SRP)**: Le code de création est séparé du code métier.

Les deux variantes de « Factory »

- Le patron de conception **Factory** présente principalement **deux** catégories :
 1. Factory Method
 2. Abstract Factory
- Chacune des deux catégories a des besoins spécifiques dans la création d'objets:

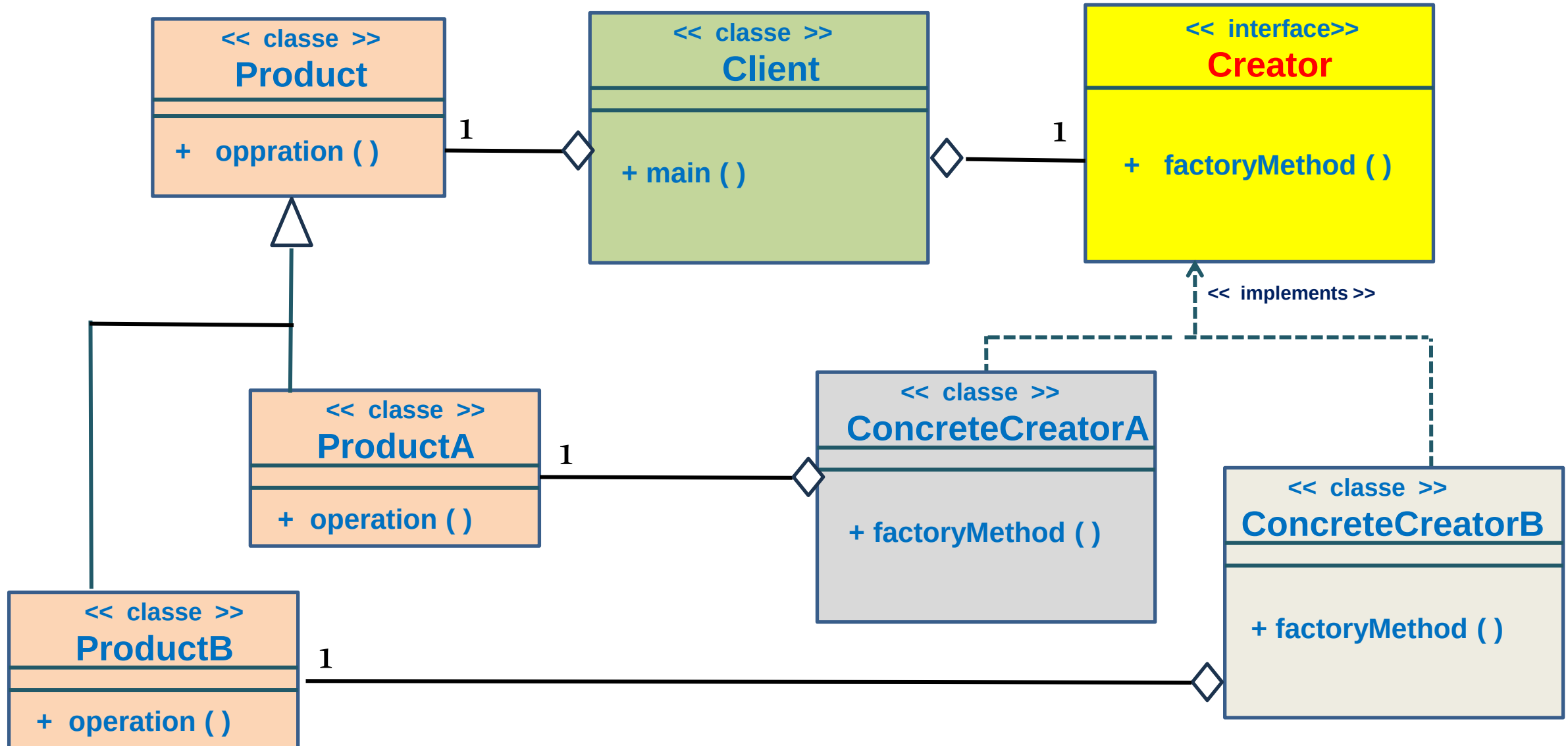
1. Factory Method

- Définit une interface pour créer un objet, mais laisse les sous-classes choisir la classe concrète à instancier.
- Il est utilisé quand une classe ne sait pas à l'avance quel type exact d'objet elle doit créer.

• Abstract Factory

- Permet de créer des familles d'objets liés sans spécifier leurs classes concrètes.
- Il est utilisé lorsqu'on veut s'assurer que des objets compatibles sont créés ensemble.

Modélisation UML –Factory Method



Implémentation JAVA- Factory Method

```
//structure de Produit de base à créer
public interface Notification
{
    void envoyer(String message);
}
```

```
//premier produit concret : EmailNotification
public class EmailNotification implements Notification{
    @Override
    public void envoyer(String message) {
        System.out.println("Envoi de l'email : " + message);
    }
}
```

```
//deuxieme produit concret : SMSNotification
public class SMSNotification implements Notification {
    @Override
    public void envoyer(String message) {
        System.out.println("Envoi du SMS : " + message);
    }
}
```

```
//Factory de notification
public abstract class NotificationFactory {
    //la méthode fabrique - factory method
    public abstract Notification createNotification();
}
```

```
//Première Factory de notification concrète
public class EmailNotificationFactory extends
NotificationFactory
{
    @Override
    public Notification createNotification()
    {
        return new EmailNotification();
    }
}
```

```
//Deuxième Factory de notification concrète
public class SMSNotificationFactory extends
NotificationFactory
{
    @Override
    public Notification createNotification()
    {
        return new SMSNotification();
    }
}
```

Implémentation JAVA- Factory Method

```
// Application Cliente
public class Client {
    public static void main(String[] args) {
        NotificationFactory factory; // déclarer une factory
        Scanner s = new Scanner(System.in);
        System.out.println("Donner un type de notification:");
        String type = s.next(); // Choix dynamique du type de notification
        switch (type) {
            case "EMAIL":
                factory = new EmailNotificationFactory();
                break;
            case "SMS":
                factory = new SMSNotificationFactory();
                break;
            default:
                throw new IllegalArgumentException("Type inconnu");
        }
        // Création de notification
        Notification notification = factory.createNotification();
        // Réalisation de l'opération
        notification.envoyer(" voilà notre premier exemple avec le DP Factory Method!");
    }
}
```

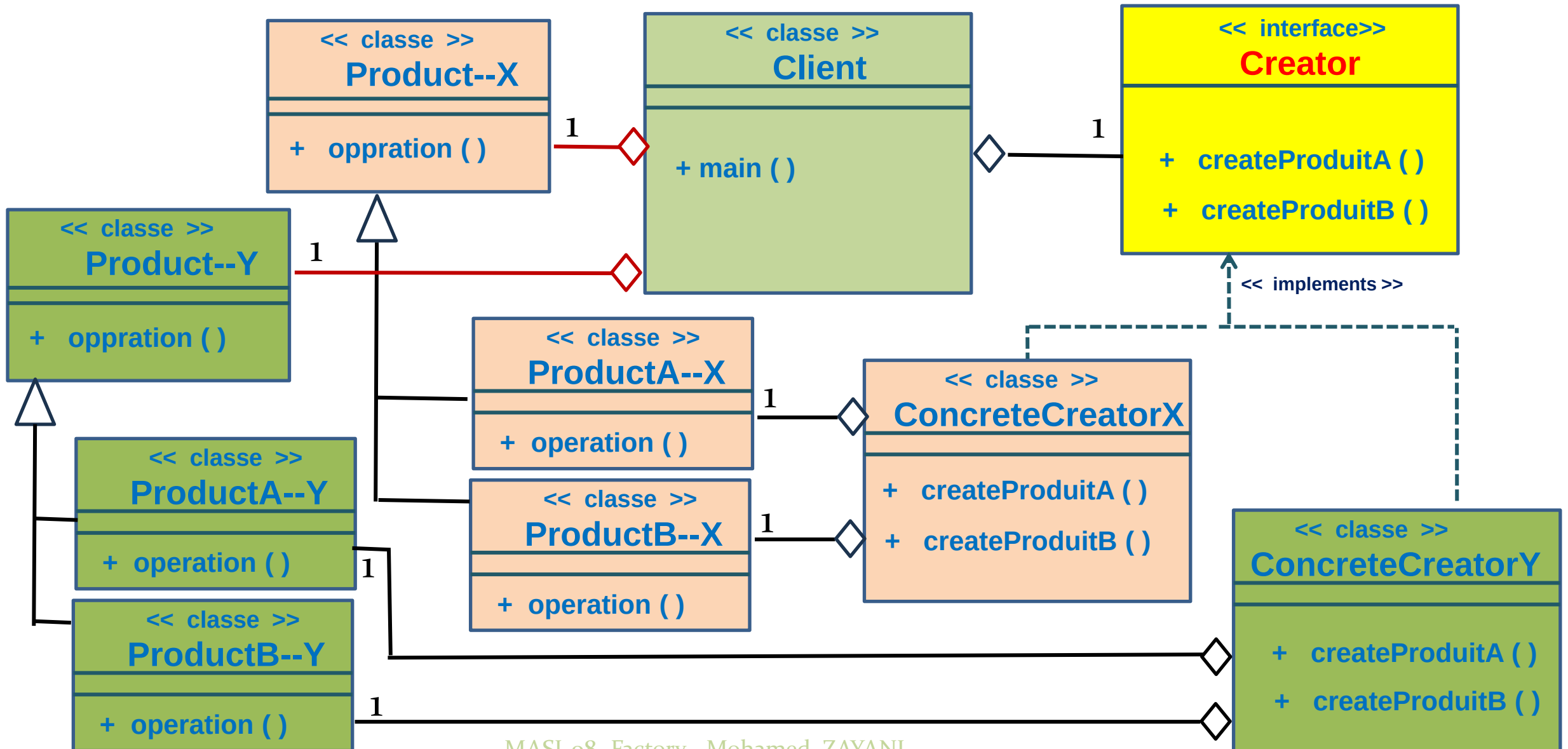
- Le client (Main) ne connaît pas la classe concrète
- Il interagit uniquement avec la factory method abstraite dans le créateur.
- Le choix du produit concret est délégué au créateur concret.

Utilisation de « Abstract Factory »

- L'exemple de notification vu avec le patron de conception « Factory Method » peut très bien être étendu pour illustrer un usage du patron de conception « Abstract Factory »:
- Il est possible de gérer **plusieurs canaux de notification (SMS, Email,...)** mais pour chaque canal, on associe plusieurs types de messages:
 - ✓ alerte,
 - ✓ confirmation,
 - ✓ rappel,
- Ainsi, pour chaque canal de notification, on crée une famille d'objets pour les types de messages correspondants:
- **Exemple:**
 - Canal SMS: peut envoyer plusieurs types de messages:
 - AlerteSMS
 - ConfirmationSMS
 - RappelSMS
 - Canal Email: peut envoyer plusieurs types de messages:
 - AlerteEmail
 - ConfirmationEmail
 - RappelEmail

➡ **Abstract Factory permet de créer des familles de messages liées à un canal, sans spécifier leurs classes concrètes.**

Modélisation UML – AbstractFactory



Implémentation JAVA- Abstract Factory

1. Interface pour le type de message « Alerte »

```
//structure de base d'un message «Alerte»  
public interface Alerte  
{  
    void envoyer(String message);  
}
```

• Message « Alerte » pour le canal SMS

```
public class AlerteSMS implements Alerte  
{  
    public void envoyer(String message)  
    {  
        System.out.println("SMS - Alerte : " + message);  
    }  
}
```

• Message « Alerte » pour le canal Email

```
public class AlerteEmail implements Alerte  
{  
    public void envoyer(String message)  
    {  
        System.out.println("Email - Alerte : " + message);  
    }  
}
```

2. Interface pour le type de message « Confirmation »

```
//structure de base d'un message «Confirmation»  
public interface Confirmation  
{  
    void envoyer(String message);  
}
```

• Message « Confirmation » pour le canal SMS

```
public class ConfirmationSMS implements Confirmation  
{  
    public void envoyer(String message)  
    {  
        System.out.println("SMS-Confirmation:" + message);  
    }  
}
```

• Message « Confirmation » pour le canal Email

```
public class ConfirmationEmail implements Confirmation  
{  
    public void envoyer(String message)  
    {  
        System.out.println("Email-Confirmation" + message);  
    }  
}
```

Implémentation JAVA- Abstract Factory

3. Interface pour la factory des notifications

```
//Factory de notifications (messages)
public interface NotificationFactory
{
    Alerte creerAlerte();
    Confirmation creerConfirmation();
}
```

• Factory de notifications (messages) pour le canal SMS

```
public class SMSNotificationFactory
implements NotificationFactory
{
    public Alerte creerAlerte()
    {
        return new AlerteSMS();
    }
    public Confirmation creerConfirmation()
    {
        return new ConfirmationSMS();
    }
}
```

• Factory de notifications (messages) pour le canal Email

```
public class EmailNotificationFactory
implements NotificationFactory
{
    public Alerte creerAlerte()
    {
        return new AlerteEmail();
    }
    public Confirmation creerConfirmation()
    {
        return new ConfirmationEmail();
    }
}
```

Implémentation JAVA- Abstract Factory

4. Classe Client

```
public class Client {
    public static void main(String[] args)
    {
        Scanner s = new Scanner(System.in);
        Confirmation confirmation;
        Alerte alerte;
        NotificationFactory factory;
        System.out.println("Donner un type de notification:");
        String type = s.next(); // Choix dynamique du type de notification

        switch (type)
        {
            case "EMAIL":
                factory = new EmailNotificationFactory();
                break;
            case "SMS":
                factory = new SMSNotificationFactory();
                break;
            default:
                throw new IllegalArgumentException("Type inconnu");
        }

        alerte = factory.creerAlerte();
        confirmation = factory.creerConfirmation();
        alerte.envoyer("Problème critique détecté !");
        confirmation.envoyer("Votre action a été confirmée.");
    }
}
```